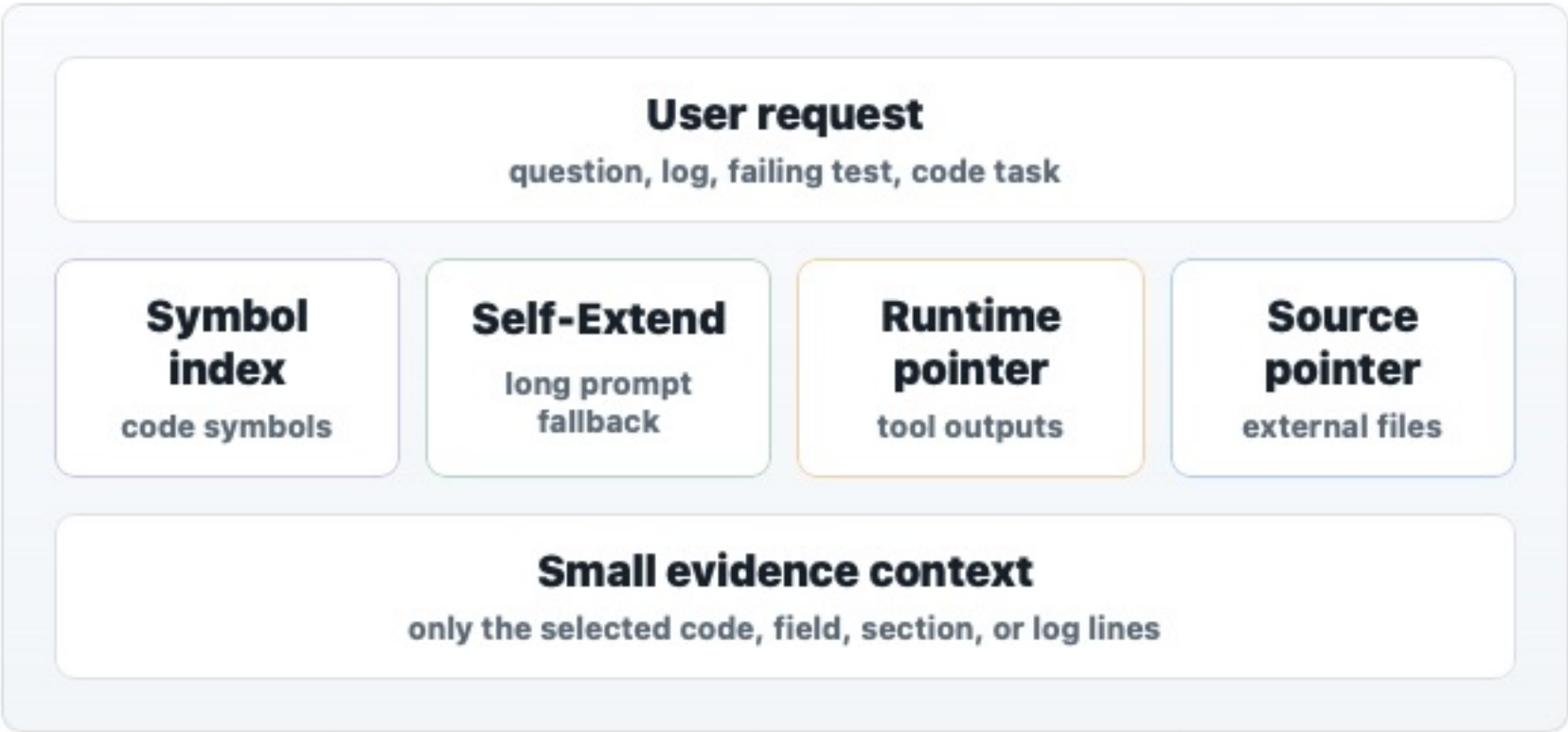


RUNTIME POINTER / SELF-EXTEND / SOURCE POINTER / SYMBOL INDEX

Experiment Results for Local LLM Context Access

A concise comparison of four ways to keep local code assistance useful when raw code, logs, documents, JSON, and tool output exceed the practical prompt budget.

- Qwen2.5 Coder 7B Q5_K_M
- Llama-2-7B-chat Q4/Q5
- llama.cpp on M3 Pro



FINAL PRODUCT

The target is a pointer-first local code assistant

Large data stays outside the prompt. The LLM receives compact addresses, chooses tools, and gets only the evidence needed for reasoning or code edits.

Design target

Use context as a workbench, not as storage. Store raw objects elsewhere and move only the relevant slice into the model prompt.

4

access strategies tested

3

visualizers already implemented

32k

main Qwen context setting

request -> candidate retrieval -> resolver / extractor -> compact evidence -> LLM answer or patch

PROBLEM

Why raw context injection fails

Context overflow

Large logs and JSON exceeded the active 32k context even at small scale.

Prompt evaluation cost

Medium symbol full repo needed 67,188 tokens and about 613 seconds for one case.

Distractor errors

Even when raw data fits, irrelevant nearby values can pull the model to the wrong answer.

Long-context retrieval is separate

Accepting a longer prompt does not guarantee the model can recover the needed fact.

Raw Prompt Token Pressure



Log scale. Lower is easier to fit in context.

FOUR SOLUTIONS

Four approaches and their visualizers

symbol_index

Build stable symbol addresses from source code, then read selected functions, classes, constants, and helpers.

memory://symbol bundle

self_extend

Use group attention to accept longer prompts when retrieval narrowing is not enough.

G=2 G=4

runtime_pointer

Store large tool outputs in RuntimeStore and pass only runtime addresses between tool calls.

runtime:// mirrored tools

source_pointer

Keep disk files outside the prompt and extract log lines, document sections, or JSON paths by pointer.

source:// typed extractors

Pointer Data Visualizer

Covers both source_pointer and runtime_pointer tracks.

Open visualizer

Self-Extend Visualizer

Shows overflow, grouped positions, and retrieval verification risk.

Open visualizer

Symbol Bundle Visualizer

Shows parser indexing, filtered selection, URI resolution, and bundle context.

Open visualizer

SOLUTION 1

symbol_index

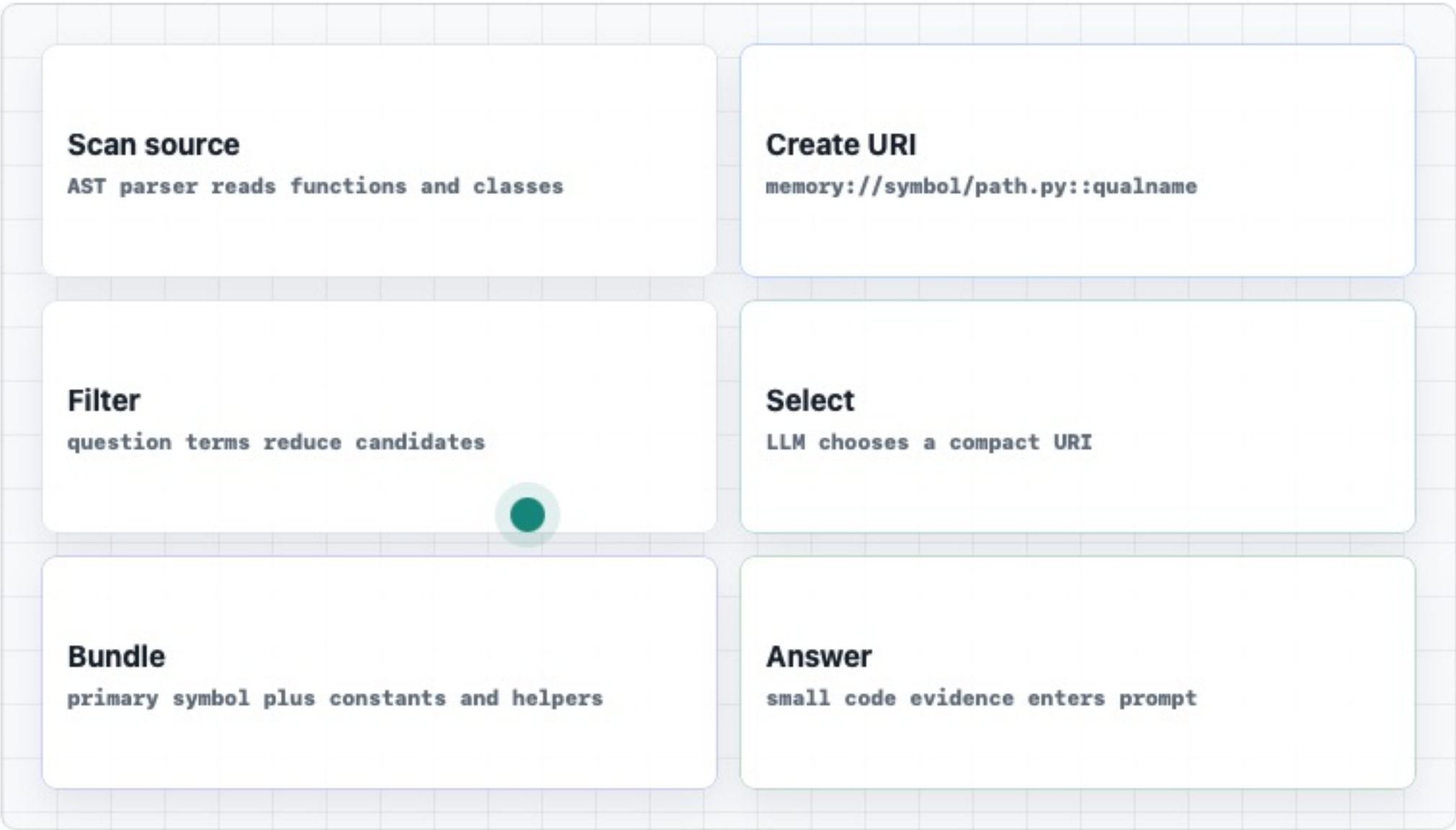
Parse the repository into compact symbol records.
Resolve selected URIs into code bodies or
dependency bundles only when needed.

AST parser

memory://symbol/file.py::qualname

filtered bundle

Open symbol visualizer



Symbol Index -> Filtered Bundle

This simplified view shows the essential path only: local source is indexed into symbol addresses, the question filters candidates, one URI is resolved, and a local BundleBuilder packs the filtered context.

LEFTPREVIOUSRIGHTNEXT

QUESTION

What exact literal is raised for an expired token?

What does build_cache_key return?

What are the invoice status and late fee?

01
Scan

02
Index

03
Terms

04
Filter

05
Select

06
Resolve

07
Bundle

services/auth_service.pyopened

reports/cache.pyindexed

billing/invoice.pyindexed

services/auth_service.py

1TOKEN_EXPIRED_MESSAGE = "token expired"

2TOKEN_REVOKED_MESSAGE = "token revoked"

3

13def parse_bearer_token(header):

14 prefix, token = header.split(" ", 1)

15 return token if prefix == "Bearer" else None

16

20def validate_token(token, clock):

21 if token.revoked:

22 raise ValueError(TOKEN_REVOKED_MESSAGE)

23 if token.expires_at < clock.now():

24 raise ValueError(TOKEN_EXPIRED_MESSAGE)

25 return token.user_id

A local BundleBuilder creates the bundle from the resolved primary symbol, referenced names, same-file callees, and symbols mentioned in the question.

validate_tokenexpiredTOKEN_EXPIRED_MESSAGE

Symbol Index7 rows

parse_bearer_tokenfunction | services/auth_service...memory://symbol/services/auth_se...

validate_tokenfunction | services/auth_service...memory://symbol/services/auth_se...

sanitize_report_namefunction | reports/cache.py#L3-L4memory://symbol/reports/cache.py...

build_cache_keyfunction | reports/cache.py#L6-L8memory://symbol/reports/cache.py...

InvoiceCalculatorclass | billing/invoice.py#L4-L10memory://symbol/billing/invoice....

InvoiceCalculator.compute_late_feefunction | billing/invoice.py#L8...memory://symbol/billing/invoice....

Candidate Outline2

validate_tokenfunction | score 13memory://symbol/services/auth_servi...

parse_bearer_tokenfunction | score 2memory://symbol/services/auth_servi...

Selected URImemory://symbol/services/auth_service.py::validate_token

URI:memory://symbol/services/auth_service.py::validate_token- services/aL25>

WHO

Local BundleBuilder, not the selector LLM.

HOW

Start with the resolved primary span, then scan local AST references.

CRITERIA

Keep the pr preferred callees, and symbols.

Primary: validate_tokenTOKEN_EXPIRED_MESSAGE

Primary symbolservices/aut

def validate_token(token, clock):
 if token.revoked:
 raise ValueError(TOKEN_REVOKED_MESSAGE)
 if token.expires_at < clock.now():
 raise ValueError(TOKEN_EXPIRED_MESSAGE)
 return token.user_id

Referenced module constant services/a

TOKEN_EXPIRED_MESSAGE = "token expired"

LLM Context Window

Question: What exact literal is raised for an expired token?

Selected URI: memory://symbol/services/auth_service.py::validate_token

Filtered Bundle: BundleBuilder packed 2 fragments with provenance

7

Only the filtered bundle enters context

A local BundleBuilder creates the bundle from the resolved primary symbol, referenced names, same-file callees, and symbols mentioned in the question.

Previous

Next

SOLUTION 2

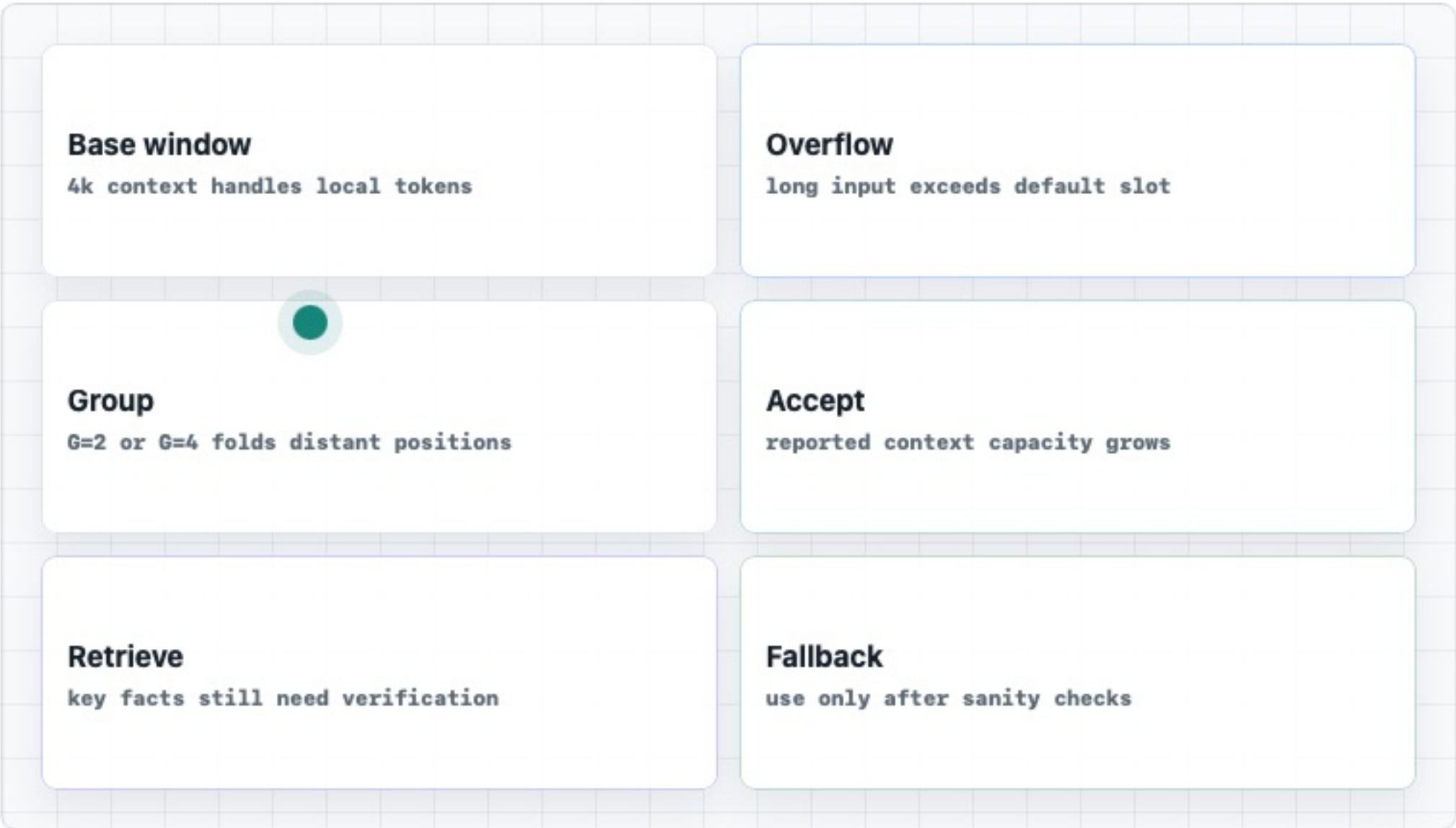
self_extend

Group attention changes inference-time position mapping so prompts beyond the normal window can be accepted.

Important distinction

It can avoid overflow, but this experiment shows that retrieval quality must still be verified before using it for code edits.

Open Self-Extend visualizer



Self-Extend Visualizer

The essential flow: base context slots overflow, Self-Extend groups distant positions, local tokens stay precise, and retrieval still needs verification.

CURRENT MODE

G=4

Effective context: ~16K

Token Stream

24 visual tokens

Prompt T01 T02 T03 T04 T05 T06 T07 T08 T09 T10 T11 T12 KEY T14 T15 T16

Base Position Grid

8 slots = trained 4K window

Overflow

0

P1 C1 P2 C2 P3 C3 P4 C4 P5 C5 P6 T19 P7 T20 P8 T21

Clear

Grouped Position Layer

4 tokens per coarse position

Group 1 C1 T01 T02 T03 T04 Group 2 C2 T05 T06 T07 T08 Group 3 C3 T09 T10 T11 T12 Group 4 C4 KEY T14 T15 Group 5 C5

Local Neighbor Window

High resolution

T19 T20 T21 T22 T23 T24

ACCEPTED BY CONTEXT

Yes

RECOVERED THE ANSWER

No

KEY IDEA

Fit and recall are separate.

STEP 6 OF 6

Retrieval must still be verified

SAY THIS

The passkey is inside the accepted prompt, but it sits in the compressed distant region. The model can still fail to recover it.

POINT AT

Point at the KEY token, then at Recovered the answer: No.

Self-Extend can expand the accepted context length, but the model still needs retrieval verification before using the result for code assistance.

← Previous

Restart →

Use ← and → arrow keys during the demo.

● ● ● ● ● ●

SOLUTION 3

runtime_pointer

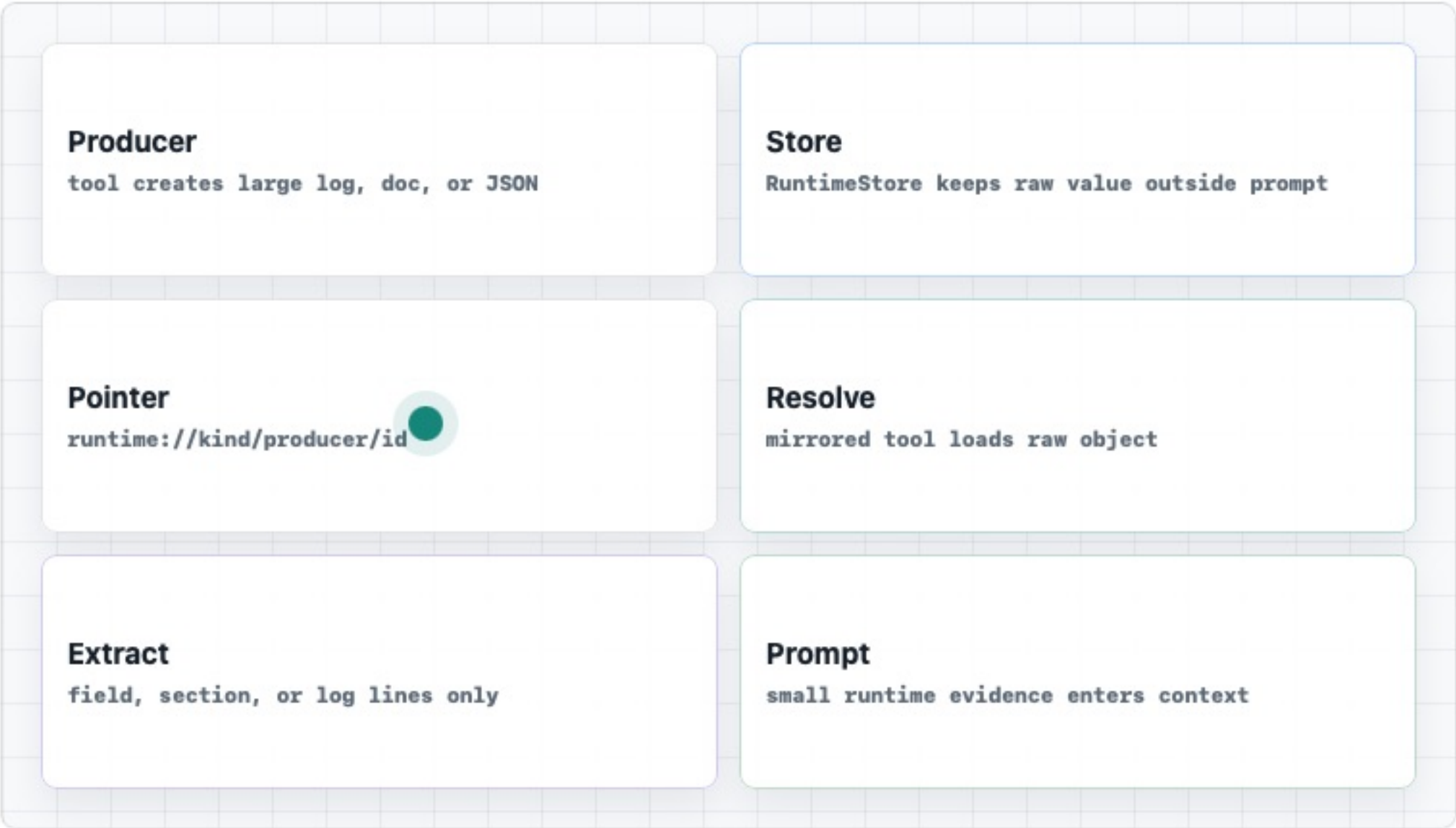
A producer tool stores large intermediate values in RuntimeStore. Later tools receive a `runtime://` address and resolve only the needed part.

producer output

RuntimeStore

mirrored extraction

Open pointer visualizer



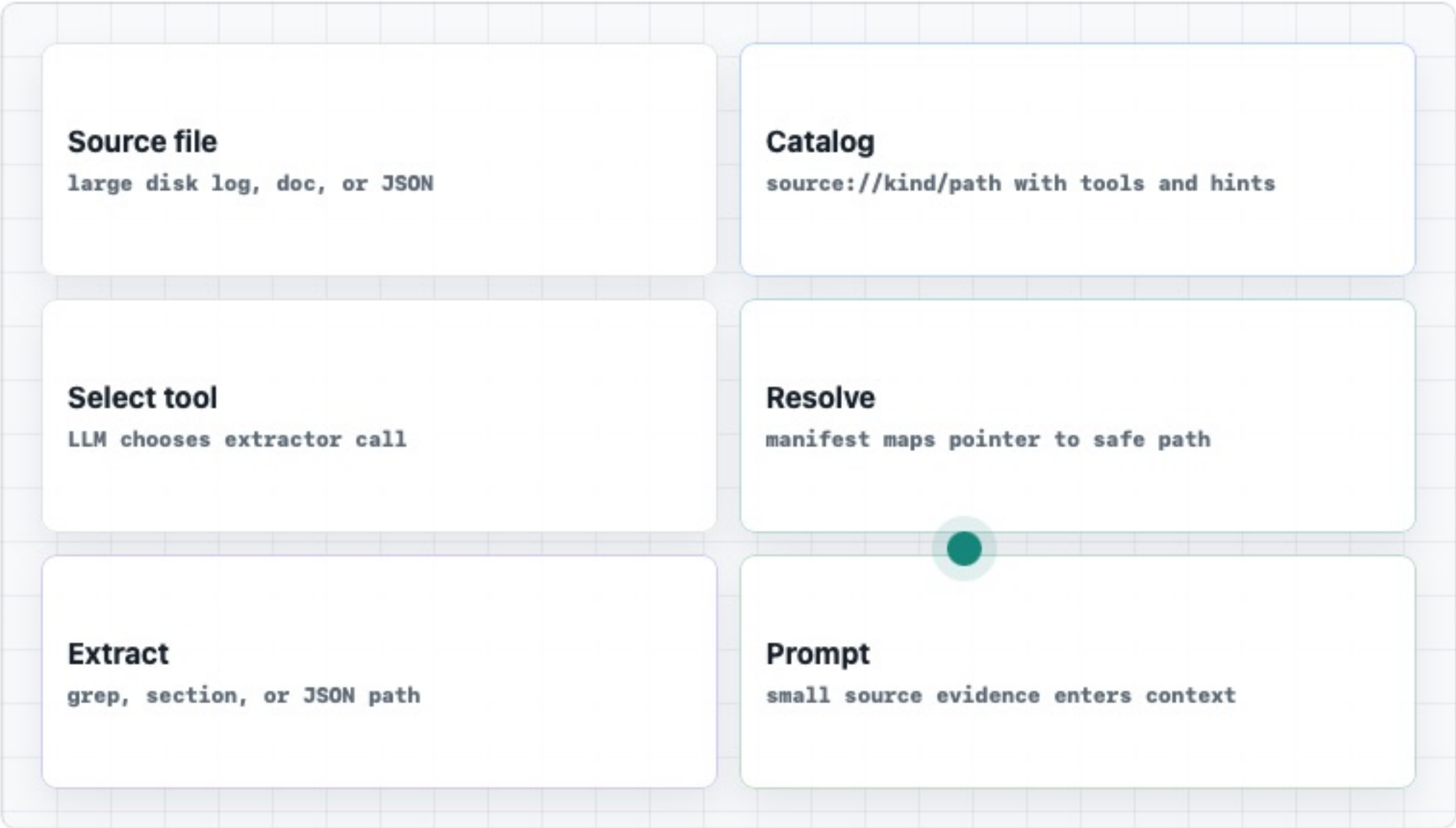
SOLUTION 4

source_pointer

Disk files stay in a safe source root. The LLM sees a catalog entry and invokes format-specific tools for logs, markdown, or JSON.

- grep_log
- extract_doc_section
- extract_json_field

Open pointer visualizer



Pointer Data Visualizer

This interactive model shows how short source and runtime pointers keep large data outside the prompt, then resolve through typed extraction tools to send only targeted evidence into the LLM context.

Closed Network

Raw Data Stays Outside

Typed Extraction Tools

Source Domain

Disk Files

Large Source Object

log file

09:17:08 INFO checkout request accepted
09:17:09 INFO tenant **Source Pointer** ded
09:17:10 WARN retry
09:17:11 ERROR payment gateway timeout
09:17:11 INFO fallback queue scheduled
09:17:12 INFO checkout trace closed

Raw source data is cataloged, not copied into the prompt.

Source Manifest

source:///kind/path

POINTER	KIND	BYTES
source:///log/logs/checkout_gateway.log	log	18 KB
source:///doc/docs/ops_runbook.md	doc	24 KB
source:///json/json/tenant_snapshot.json	json	31 KB

Source Pointer

Points to existing external data on disk through a safe catalog entry, without exposing the real filesystem path to the LLM.

Source Ready

A disk file is available, but only a safe source catalog entry will be exposed.

Pointer Catalog

Compact Exposure

SOURCE ENTRY

18 KB

source:///log/logs/checkout_gateway.log
tools: grep_log
hints: checkout, gateway, ERROR

RUNTIME ENTRY

42 KB

runtime:///log/collect_checkout_trace/0001
tools: grep_runtime_log
hints: session, timeout, trace

Source route: disk object to source catalog.

Runtime route: produced value to RuntimeStore catalog.

Active flow: selected pointer through tools into context.

Source Pointer

Runtime Pointer

Log

Document

JSON

1 Raw Data Object

2 Pointer Issued

3 Catalog Exposed

4 Tool Selected

5 Pointer Resolved

6 Evidence Extracted

7 Prompt Assembled

8 LLM Context

LLM Selector Output

tool command

{
 "tool": null,
 "args": {}
}

Resolver Detail View

Source

Runtime

Pointer

source:///log/l

Manifest Lookup

Find catalog row and allowed source root.

Concrete Data Location

sources_root/l

Format-Specific Extraction

waiting

Log Extraction

41 WARN retry policy selected
42 ERROR payment gateway timeout
43 INFO fallback queue scheduled

Document Extraction

Normal Flow

Gateway Timeout

Follow-up

JSON Extraction

tenant
tenant.limits
tenant.lim

LLM Context Window

assembled prompt

Pointer and Tool Call

No pointer or tool call has entered the prompt yet.

Extracted Evidence Only

Raw data is excluded. Evidence will appear after extracti

Runtime Domain

RuntimeStore

Large Runtime Object

log value

tool: collect_checkout_trace
session: local-run-7
TRACE request started **Runtime Pointer**
TRACE provider call started
ERROR provider timeout after 2940 ms
TRACE fallback queue created

Raw runtime output is stored, then represented by a short pointer.

RuntimeStore

runtime:///kind/producer/id

POINTER	PRODUCER
runtime:///log/collect_checkout_trace/0001	collect_checkout
runtime:///doc/generate_runbook_summary/0002	generate_runbook
runtime:///json/load_tenant_snapshot/0003	load_tenant_snap

Runtime Pointer

Points to a value created during the current tool run and stored in RuntimeStore, so the value can move between tools without entering the prompt.

Runtime Ready

A producer tool can create a large value, but the LLM will only receive a pointer.

→ Next Step

▶ Auto Play

↺ Reset

0 Ready State

Choose a pointer track and data format, then step through how raw objects stay outside the prompt.

CONTEXT FILL

Pointer Data Visualizer

This interactive model shows how short source and runtime pointers keep large data outside the prompt, then resolve through typed extraction tools to send only targeted evidence into the LLM context.

Closed Network

Raw Data Stays Outside

Typed Extraction Tools

Source Domain

Disk Files

Large Source Object

log file

09:17:08 INFO checkout request accepted
09:17:09 INFO tenant_id=acme-42 loaded
09:17:10 WARN retry policy selected
09:17:11 ERROR payment gateway timeout
09:17:11 INFO fallback queue scheduled
09:17:12 INFO checkout trace closed

Raw source data is cataloged, not copied into the prompt.

Source Manifest

source://kind/path

POINTER	KIND	BYTES
source://log/logs/checkout_gateway.log	log	18 KB
source://doc/docs/ops	doc	24 KB
source://json/json/tenant_snapshot.json	json	31 KB

Source Pointer

Source Pointer

Points to existing external data on disk through a safe catalog entry, without exposing the real filesystem path to the LLM.

Source Pointer Issued

The manifest creates a source://kind/path address for the selected disk object.

Pointer Catalog

Compact Exposure

SOURCE ENTRY

18 KB

source://log/logs/checkout_gateway.log
tools: grep_log
hints: checkout, gateway, ERROR

RUNTIME ENTRY

42 KB

runtime://log/collect_checkout_trace/0001
tools: grep_runtime_log
hints: session, timeout, trace

Source route: disk object to source catalog.

Runtime route: produced value to RuntimeStore catalog.

Active flow: selected pointer through tools into context.

Source Pointer

Runtime Pointer

Log

Document

JSON

1 Raw Data Object

2 Pointer Issued

3 Catalog Exposed

4 Tool Selected

5 Pointer Resolved

6 Evidence Extracted

7 Prompt Assembled

8 LLM Context

LLM Selector Output

tool command

{
 "tool": null,
 "args": {}
}

Resolver Detail View

Source

Runtime

Pointer

source://log/l

Manifest Lookup

Find catalog row and allowed source root.

Concrete Data Location

sources_root/l

Format-Specific Extraction

waiting

Log Extraction

41 WARN retry policy selected
42 ERROR payment gateway timeout
43 INFO fallback queue scheduled

Document Extraction

Normal Flow
Gateway Timeout
Follow-up

JSON Extraction

tenant
tenant.limits
tenant.lim

LLM Context Window

assembled prompt

Pointer and Tool Call

No pointer or tool call has entered the prompt yet.

Extracted Evidence Only

Raw data is excluded. Evidence will appear after extracti

Runtime Domain

RuntimeStore

Large Runtime Object

log value

tool: collect_checkout_trace
session: local-run-7
TRACE request started
TRACE provider call started
ERROR provider timeout after 2940 ms
TRACE fallback queue created

Raw runtime output is stored, then represented by a short pointer.

RuntimeStore

runtime://kind/producer/id

POINTER	PRODUCER
runtime://log/collect_checkout_trace/0001	collect_checkout
runtime://doc/generate_runbook	generate_runbook
runtime://json/load_tenant_snapshot/0003	load_tenant_snap

Runtime Pointer

Points to a value created during the current tool run and stored in RuntimeStore, so the value can move between tools without entering the prompt.

Runtime Pointer Issued

RuntimeStore creates a runtime://kind/producer/id address for the stored value.

Next Step

Auto Play

Reset

2

Step 2: A Short Pointer Is Issued

The selected domain creates a compact pointer that can move through the system instead of the large data value.

CONTEXT FILL

Pointer Data Visualizer

This interactive model shows how short source and runtime pointers keep large data outside the prompt, then resolve through typed extraction tools to send only targeted evidence into the LLM context.

Closed Network

Raw Data Stays Outside

Typed Extraction Tools

Source Domain

Disk Files

Large Source Object

log file

09:17:08 INFO checkout request accepted
09:17:09 INFO tenant_id=acme-42 loaded
09:17:10 WARN retry policy selected
09:17:11 ERROR payment gateway timeout
09:17:11 INFO fallback queue scheduled
09:17:12 INFO checkout trace closed

Raw source data is cataloged, not copied into the prompt.

Source Manifest

source://kind/path

POINTER	KIND	BYTES
source://log/logs/checkout_gateway.log	log	18 KB
source://doc/docs/ops_runbook.md	doc	24 KB
source://json/json/tenant_snapshot.json	json	31 KB

Source Pointer

Points to existing external data on disk through a safe catalog entry, without exposing the real filesystem path to the LLM.

Source Catalog Row

The LLM sees pointer, kind, byte size, available tools, and hints only.

Pointer Catalog

Compact Exposure

SOURCE ENTRY

18 KB

source://log/logs/checkout_gateway.log

tools: grep_log

hints: checkout, gateway, ERROR

SOURCE Pointer

RUNTIME ENTRY

42 KB

runtime://log/collect_checkout_trace/0001

tools: grep_runtime_log

hints: session, timeout, trace

RUNTIME Pointer

Source route: disk object to source catalog.

Runtime route: produced value to RuntimeStore catalog.

Active flow: selected pointer through tools into context.

Source Pointer

Runtime Pointer

Log

Document

JSON

1 Raw Data Object

Large source or runtime data exists outside the prompt.

2 Pointer Issued

A short address token represents the large object.

3 Catalog Exposed

Only pointer metadata is shown to the LLM.

4 Tool Selected

The LLM chooses a typed extraction tool.

5 Pointer Resolved

Resolver maps the pointer to a concrete location.

6 Evidence Extracted

Only the needed slice is returned.

7 Prompt Assembled

Question, tool call, and evidence are packed.

8 LLM Context

The compact payload enters the context window.

LLM Selector Output

tool command

{
 "tool": null,
 "args": {}
}

Resolver Detail View

Source

Runtime

Pointer source://log/l

Manifest Lookup Find catalog row and allowed source root.

Concrete Data Location sources_root/l

Format-Specific Extraction

waiting

Log Extraction

41 WARN retry policy selected

42 ERROR payment gateway timeout

43 INFO fallback queue scheduled

Document Extraction

Normal Flow

Gateway Timeout

Follow-up

JSON Extraction

tenant

tenant.limits

tenant.lim

LLM Context Window

assembled prompt

Pointer and Tool Call

No pointer or tool call has entered the prompt yet.

Extracted Evidence Only

Raw data is excluded. Evidence will appear after extracti

Runtime Domain

RuntimeStore

Large Runtime Object

log value

tool: collect_checkout_trace
session: local-run-7
TRACE request started
TRACE provider call started
ERROR provider timeout after 2940 ms
TRACE fallback queue created

Raw runtime output is stored, then represented by a short pointer.

RuntimeStore

runtime://kind/producer/id

POINTER	PRODUCER
runtime://log/collect_checkout_trace/0001	collect_checkout
runtime://doc/generate_runbook_summary/0002	generate_runbook
runtime://json/load_tenant_snapshot/0003	load_tenant_snap

Runtime Pointer

Points to a value created during the current tool run and stored in RuntimeStore, so the value can move between tools without entering the prompt.

Runtime Catalog Row

The LLM sees a compact runtime address card with tools and hints, not the value body.

Next Step

Auto Play

Reset

3

Step 3: The Compact Catalog Is Exposed

The LLM sees pointer, kind, byte size, tools, and hints, but not the raw source file or raw runtime output.

CONTEXT FILL

Pointer Data Visualizer

This interactive model shows how short source and runtime pointers keep large data outside the prompt, then resolve through typed extraction tools to send only targeted evidence into the LLM context.

Closed Network

Raw Data Stays Outside

Typed Extraction Tools

Source Domain

Disk Files

Large Source Object

log file

09:17:08 INFO checkout request accepted
09:17:09 INFO tenant_id=acme-42 loaded
09:17:10 WARN retry policy selected
09:17:11 ERROR payment gateway timeout
09:17:11 INFO fallback queue scheduled
09:17:12 INFO checkout trace closed

Raw source data is cataloged, not copied into the prompt.

Source Manifest

source://kind/path

POINTER	KIND	BYTES
source://log/logs/checkout_gateway.log	log	18 KB
source://doc/docs/ops_runbook.md	doc	24 KB
source://json/json/tenant_snapshot.json	json	31 KB

Source Pointer

Points to existing external data on disk through a safe catalog entry, without exposing the real filesystem path to the LLM.

Source Tool Choice

A source extraction tool is selected, such as grep_log or extract_doc_section.

Pointer Catalog

Compact Exposure

SOURCE ENTRY

18 KB

source://log/logs/checkout_gateway.log
tools: grep_log
hints: checkout, gateway, ERROR

RUNTIME ENTRY

42 KB

runtime://log/collect_checkout_trace/0001
tools: grep_runtime_log
hints: session, timeout, trace

Source route: disk object to source catalog.

Runtime route: produced value to RuntimeStore catalog.

Active flow: selected pointer through tools into context.

Source Pointer

Runtime Pointer

Log

Document

JSON

1 Raw Data Object

2 Pointer Issued

3 Catalog Exposed

4 Tool Selected

5 Pointer Resolved

6 Evidence Extracted

7 Prompt Assembled

8 LLM Context

LLM Selector Output

tool command

Source Pointer

Runtime Pointer

{
 "tool": "grep_log",
 "arg": "checkout_gateway.log",
 "pattern": "ERROR",
 "before": 1,
 "after": 1
}

Format-Specific Extraction

grep_log

Log Extraction

41 WARN retry policy selected
42 ERROR payment gateway timeout
43 INFO fallback queue scheduled

Document Extraction

Normal Flow
Gateway Timeout
Follow-up

JSON Extraction

tenant
tenant.limits
tenant.lim

Resolver Detail View

Source

Runtime

Pointer

source://log/l

Manifest Lookup

Find catalog row and allowed source root.

Concrete Data Location

sources_root/l

LLM Context Window

assembled prompt

Pointer and Tool Call

No pointer or tool call has entered the prompt yet.

Extracted Evidence Only

Raw data is excluded. Evidence will appear after extracti

Runtime Domain

RuntimeStore

Large Runtime Object

log value

tool: collect_checkout_trace
session: local-run-7
TRACE request started
TRACE provider call started
ERROR provider timeout after 2940 ms
TRACE fallback queue created

Raw runtime output is stored, then represented by a short pointer.

RuntimeStore

runtime://kind/producer/id

POINTER	PRODUCER
runtime://log/collect_checkout_trace/0001	collect_checkout
runtime://doc/generate_runbook_summary/0002	generate_runbook
runtime://json/load_tenant_snapshot/0003	load_tenant_snap

Runtime Pointer

Points to a value created during the current tool run and stored in RuntimeStore, so the value can move between tools without entering the prompt.

Runtime Tool Choice

A runtime extraction tool is selected, such as grep_runtime_log or extract_runtime_json_path.

Next Step

Auto Play

Reset

4

Step 4: The LLM Selects a Typed Extraction Tool

The selector combines the user question, tool specs, and catalog row into a JSON-like command card.

CONTEXT FILL

Pointer Data Visualizer

This interactive model shows how short source and runtime pointers keep large data outside the prompt, then resolve through typed extraction tools to send only targeted evidence into the LLM context.

Closed Network

Raw Data Stays Outside

Typed Extraction Tools

Source Domain

Disk Files

Large Source Object

log file

```
09:17:08 INFO checkout request accepted
09:17:09 INFO tenant_id=acme-42 loaded
09:17:10 WARN retry policy selected
09:17:11 ERROR payment gateway timeout
09:17:11 INFO fallback queue scheduled
09:17:12 INFO checkout trace closed
```

Raw source data is cataloged, not copied into the prompt.

Source Manifest

source://kind/path

POINTER	KIND	BYTES
source://log/logs/checkout_gateway.log	log	18 KB
source://doc/docs/ops_runbook.md	doc	24 KB
source://json/json/tenant_snapshot.json	json	31 KB

Source Pointer

Points to existing external data on disk through a safe catalog entry, without exposing the real filesystem path to the LLM.

Source Resolve

The resolver maps source://kind/path through the manifest into sources_root/rel_path.

Pointer Catalog

Compact Exposure

SOURCE ENTRY

18 KB

source://log/logs/checkout_gateway.log
tools: grep_log
hints: checkout, gateway, ERROR

Source route: disk object to source catalog.

RUNTIME ENTRY

42 KB

runtime://log/collect_checkout_trace/0001
tools: grep_runtime_log
hints: session, timeout, trace

Runtime route: produced value to RuntimeStore catalog.

Active flow: selected pointer through tools into context.

Source Pointer

Runtime Pointer

Log

Document

JSON

1

Raw Data Object

Large source or runtime data exists outside the prompt.

2

Pointer Issued

A short address token represents the large object.

3

Catalog Exposed

Only pointer metadata is shown to the LLM.

4

Tool Selected

The LLM chooses a typed extraction tool.

5

Pointer Resolved

Resolver maps the pointer to a concrete location.

6

Evidence Extracted

Only the needed slice is returned.

7

Prompt Assembled

Question, tool call, and evidence are packed.

8

LLM Context

The compact payload enters the context window.

LLM Selector Output

tool command

```
{
  "tool": "grep_log",
  "args": {
    "pointer": "source://log/logs/checkout_gateway.log",
    "pattern": "ERROR",
    "before": 1,
    "after": 1
  }
}
```

Resolver Detail View

Source Runtime

Pointer

source://log/logs/checkout_gateway.log

Source Pointer

source://log/logs/checkout_gateway.log

Runtime Pointer

runtime://log/collect_checkout_trace/0001

Concrete Data Location

sources_root/log/logs/checkout_gateway.log

Format-Specific Extraction

grep_log

Log Extraction

```
41  WARN
    retry
    policy
    selected
42  ERROR
    payment
    gateway
    timeout
43  INFO
    fallback
    queue
    scheduled
```

Document Extraction

Normal Flow
Gateway Timeout
Follow-up

JSON Extraction

```
tenant
tenant.limits
tenant.limits
```

LLM Context Window

assembled prompt

Pointer and Tool Call

No pointer or tool call has entered the prompt yet.

Extracted Evidence Only

Raw data is excluded. Evidence will appear after extraction.

Runtime Domain

RuntimeStore

Large Runtime Object

log value

```
tool: collect_checkout_trace
session: local-run-7
TRACE request started
TRACE provider call started
ERROR provider timeout after 2940 ms
TRACE fallback queue created
```

Raw runtime output is stored, then represented by a short pointer.

RuntimeStore

runtime://kind/producer/id

POINTER	PRODUCER
runtime://log/collect_checkout_trace/0001	collect_checkout
runtime://doc/generate_runbook_summary/0002	generate_runbook
runtime://json/load_tenant_snapshot/0003	load_tenant_snapshot

Runtime Pointer

Points to a value created during the current tool run and stored in RuntimeStore, so the value can move between tools without entering the prompt.

Runtime Resolve

The resolver maps runtime://kind/producer/id to RuntimeStore and optional nested paths.

Next Step

Auto Play

Reset

5

Step 5: The Pointer Is Resolved

Source pointers resolve through the manifest to a safe source root. Runtime pointers resolve through RuntimeStore and optional nested paths.

CONTEXT FILL



Pointer Data Visualizer

This interactive model shows how short source and runtime pointers keep large data outside the prompt, then resolve through typed extraction tools to send only targeted evidence into the LLM context.

Closed Network

Raw Data Stays Outside

Typed Extraction Tools

Source Domain

Disk Files

Large Source Object

log file

```
09:17:08 INFO checkout request accepted
09:17:09 INFO tenant_id=acme-42 loaded
09:17:10 WARN retry policy selected
09:17:11 ERROR payment gateway timeout
09:17:11 INFO fallback queue scheduled
09:17:12 INFO checkout trace closed
```

Raw source data is cataloged, not copied into the prompt.

Source Manifest

source://kind/path

POINTER	KIND	BYTES
source://log/logs/checkout_gateway.log	log	18 KB
source://doc/docs/ops_runbook.md	doc	24 KB
source://json/json/tenant_snapshot.json	json	31 KB

Source Pointer

Points to existing external data on disk through a safe catalog entry, without exposing the real filesystem path to the LLM.

Source Evidence

The tool extracts only nearby log lines, one document section, or one JSON field.

Pointer Catalog

Compact Exposure

SOURCE ENTRY

18 KB

```
source://log/logs/checkout_gateway.log
tools: grep_log
hints: checkout, gateway, ERROR
```

RUNTIME ENTRY

42 KB

```
runtime://log/collect_checkout_trace/0001
tools: grep_runtime_log
hints: session, timeout, trace
```

Source route: disk object to source catalog.

Runtime route: produced value to RuntimeStore catalog.

Active flow: selected pointer through tools into context.

Source Pointer

Runtime Pointer

Log

Document

JSON

1

Raw Data Object

Large source or runtime data exists outside the prompt.

2

Pointer Issued

A short address token represents the large object.

3

Catalog Exposed

Only pointer metadata is shown to the LLM.

4

Tool Selected

The LLM chooses a typed extraction tool.

5

Pointer Resolved

Resolver maps the pointer to a concrete location.

6

Evidence Extracted

Only the needed slice is returned.

7

Prompt Assembled

Question, tool call, and evidence are packed.

8

LLM Context

The compact payload enters the context window.

LLM Selector Output

tool command

```
{
  "tool": "grep_log",
  "args": {
    "pointer": "source://log/logs/checkout_gateway.log",
    "pattern": "ERROR",
    "before": 1,
    "after": 1
  }
}
```

Resolver Detail View

Source

Runtime

```
Pointer source://log/l → Manifest Lookup Find catalog row and allowed source root. → Concrete Data Location sources_root/l
```

Format-Specific Extraction

grep_log

Log Extraction

Document

JSON

```
41
42 09:17:10 WARN retry policy selected
43 09:17:11 ERROR payment gateway timeout
09:17:11 INFO fallback queue scheduled
scheduled
```

Source Pointer

Runtime Pointer

Extracted Evidence

LLM Context Window

assembled prompt

Pointer and Tool Call

No pointer or tool call has entered the prompt yet.

Extracted Evidence Only

```
Evidence prepared outside the prompt:
lines 41-43
09:17:10 WARN retry policy selected
09:17:11 ERROR payment gateway timeout
09:17:11 INFO fallback queue scheduled
```

Runtime Domain

RuntimeStore

Large Runtime Object

log value

```
tool: collect_checkout_trace
session: local-run-7
TRACE request started
TRACE provider call started
ERROR provider timeout after 2940 ms
TRACE fallback queue created
```

Raw runtime output is stored, then represented by a short pointer.

RuntimeStore

runtime://kind/producer/id

POINTER	PRODUCER
runtime://log/collect_checkout_trace/0001	collect_checkout
runtime://doc/generate_runbook_summary/0002	generate_runbook
runtime://json/load_tenant_snapshot/0003	load_tenant_snapshot

Runtime Pointer

Points to a value created during the current tool run and stored in RuntimeStore, so the value can move between tools without entering the prompt.

Runtime Evidence

The tool extracts only the relevant runtime log lines, section, or JSON path.

Next Step

Auto Play

Reset

6

Step 6: Format-Specific Evidence Is Extracted

The extraction tool returns only a log neighborhood, a document section, or a JSON field with provenance.

CONTEXT FILL

Pointer Data Visualizer

This interactive model shows how short source and runtime pointers keep large data outside the prompt, then resolve through typed extraction tools to send only targeted evidence into the LLM context.

Closed Network

Raw Data Stays Outside

Typed Extraction Tools

Source Domain

Disk Files

Large Source Object

log file

```
09:17:08 INFO checkout request accepted
09:17:09 INFO tenant_id=acme-42 loaded
09:17:10 WARN retry policy selected
09:17:11 ERROR payment gateway timeout
09:17:11 INFO fallback queue scheduled
09:17:12 INFO checkout trace closed
```

Raw source data is cataloged, not copied into the prompt.

Source Manifest

source://kind/path

POINTER	KIND	BYTES
source://log/logs/checkout_gateway.log	log	18 KB
source://doc/docs/ops_runbook.md	doc	24 KB
source://json/json/tenant_snapshot.json	json	31 KB

Source Pointer

Points to existing external data on disk through a safe catalog entry, without exposing the real filesystem path to the LLM.

Source Prompt Part

The prompt receives the source pointer, tool call, and extracted evidence.

Pointer Catalog

Compact Exposure

SOURCE ENTRY

18 KB

```
source://log/logs/checkout_gateway.log
tools: grep_log
hints: checkout, gateway, ERROR
```

Source route: disk object to source catalog.

RUNTIME ENTRY

42 KB

```
runtime://log/collect_checkout_trace/0001
tools: grep_runtime_log
hints: session, timeout, trace
```

Runtime route: produced value to RuntimeStore catalog.

Active flow: selected pointer through tools into context.

Source Pointer

Runtime Pointer

Log

Document

JSON

1

Raw Data Object

Large source or runtime data exists outside the prompt.

2

Pointer Issued

A short address token represents the large object.

3

Catalog Exposed

Only pointer metadata is shown to the LLM.

4

Tool Selected

The LLM chooses a typed extraction tool.

5

Pointer Resolved

Resolver maps the pointer to a concrete location.

6

Evidence Extracted

Only the needed slice is returned.

7

Prompt Assembled

Question, tool call, and evidence are packed.

8

LLM Context

The compact payload enters the context window.

LLM Selector Output

tool command

```
{
  "tool": "grep_log",
  "args": {
    "pointer": "source://log/logs/checkout_gateway.log",
    "pattern": "ERROR",
    "before": 1,
    "after": 1
  }
}
```

Resolver Detail View

Source

Runtime

```
Pointer source://log/l → Manifest Lookup Find catalog row and allowed source root. → Concrete Data Location sources_root/l
```

Format-Specific Extraction

grep_log

Log Extraction

```
41 WARN retry policy selected
42 ERROR payment gateway timeout
43 INFO fallback queue scheduled
```

Document Extraction

```
Normal Flow
Gateway
Follow-up
```

JSON Extraction

```
tenant
tenant.limits
tenant.lim
```

Extracted Evidence

LLM Context Window

assembled prompt

Pointer and Tool Call

```
Question
Pointer
Tool call
```

Source Pointer

Runtime Pointer

```
09:17:10 WARN retry policy selected
09:17:11 ERROR payment gateway timeout
09:17:11 INFO fallback queue scheduled
09:17:11 INFO checkout trace closed
```

Runtime Domain

RuntimeStore

Large Runtime Object

log value

```
tool: collect_checkout_trace
session: local-run-7
TRACE request started
TRACE provider call started
ERROR provider timeout after 2940 ms
TRACE fallback queue created
```

Raw runtime output is stored, then represented by a short pointer.

RuntimeStore

runtime://kind/producer/id

POINTER	PRODUCER
runtime://log/collect_checkout_trace/0001	collect_checkout
runtime://doc/generate_runbook_summary/0002	generate_runbook
runtime://json/load_tenant_snapshot/0003	load_tenant_snapshot

Runtime Pointer

Points to a value created during the current tool run and stored in RuntimeStore, so the value can move between tools without entering the prompt.

Runtime Prompt Part

The prompt receives the runtime pointer, tool call, and extracted evidence.

Next Step

Auto Play

Reset

7

Step 7: The Prompt Is Assembled

The prompt contains the user question, selected pointer and tool call, and extracted evidence only.

CONTEXT FILL



Pointer Data Visualizer

This interactive model shows how short source and runtime pointers keep large data outside the prompt, then resolve through typed extraction tools to send only targeted evidence into the LLM context.

Closed Network

Raw Data Stays Outside

Typed Extraction Tools

Source Domain

Disk Files

Large Source Object

log file

09:17:08 INFO checkout request accepted
09:17:09 INFO tenant_id=acme-42 loaded
09:17:10 WARN retry policy selected
09:17:11 ERROR payment gateway timeout
09:17:11 INFO fallback queue scheduled
09:17:12 INFO checkout trace closed

Raw source data is cataloged, not copied into the prompt.

Source Manifest

source://kind/path

POINTER	KIND	BYTES
source://log/logs/checkout_gateway.log	log	18 KB
source://doc/docs/ops_runbook.md	doc	24 KB
source://json/json/tenant_snapshot.json	json	31 KB

Source Pointer

Points to existing external data on disk through a safe catalog entry, without exposing the real filesystem path to the LLM.

Source Context Entry

Only source evidence enters the LLM context while the disk file stays outside.

Pointer Catalog

Compact Exposure

SOURCE ENTRY

18 KB

source://log/logs/checkout_gateway.log
tools: grep_log
hints: checkout, gateway, ERROR

RUNTIME ENTRY

42 KB

runtime://log/collect_checkout_trace/0001
tools: grep_runtime_log
hints: session, timeout, trace

Source route: disk object to source catalog.

Runtime route: produced value to RuntimeStore catalog.

Active flow: selected pointer through tools into context.

Source Pointer

Runtime Pointer

Log

Document

JSON

1 Raw Data Object

2 Pointer Issued

3 Catalog Exposed

4 Tool Selected

5 Pointer Resolved

6 Evidence Extracted

7 Prompt Assembled

8 LLM Context

LLM Selector Output

tool command

{
 "tool": "grep_log",
 "args": {
 "pointer": "source://log/logs/checkout_gateway.log",
 "pattern": "ERROR",
 "before": 1,
 "after": 1
 }
}

Resolver Detail View

Source

Runtime

Pointer

source://log/l

Manifest Lookup

Find catalog row and allowed source root.

Concrete Data Location

sources_root/l

Format-Specific Extraction

grep_log

Log Extraction

41 WARN retry policy selected
42 ERROR payment gateway timeout
43 INFO fallback queue scheduled

Document Extraction

Normal Flow
Gateway Timeout
Follow-up

JSON Extraction

tenant
tenant.limits
tenant.lim

LLM Context Window

assembled prompt

Pointer and Tool Call

Question: ...
Pointer: ...
Tool call: grep_log(pointer, "ERROR", 1, 1)

Source Pointer

Runtime Pointer

Extracted Evidence

Extracted Evidence Only

lines 41-43
09:17:10 WARN retry policy selected
09:17:11 ERROR payment gateway timeout
09:17:11 INFO fallback queue scheduled

Runtime Domain

RuntimeStore

Large Runtime Object

log value

tool: collect_checkout_trace
session: local-run-7
TRACE request started
TRACE provider call started
ERROR provider timeout after 2940 ms
TRACE fallback queue created

Raw runtime output is stored, then represented by a short pointer.

RuntimeStore

runtime://kind/producer/id

POINTER	PRODUCER
runtime://log/collect_checkout_trace/0001	collect_checkout
runtime://doc/generate_runbook_summary/0002	generate_runbook
runtime://json/load_tenant_snapshot/0003	load_tenant_snap

Runtime Pointer

Points to a value created during the current tool run and stored in RuntimeStore, so the value can move between tools without entering the prompt.

Runtime Context Entry

Only runtime evidence enters the LLM context while the stored value remains outside.

EXPERIMENT RESULTS 1

symbol_index results

Easy tasks show the cost benefit of symbol access. Hard tasks show why single-symbol reads must expand into bundles.

Compared modes

full_repo
Insert the whole repository into the prompt.

full_file
Assume the target file is already known.

line_span
Read the exact target line range.

symbol_oracle
Read the known target symbol only.

symbol_select
Let the LLM choose from the full symbol index.

filtered_select_bundle
Filter candidates first, then expand constants and helpers.

Easy Task Wall Time

full_repo		613 s
symbol_select		181 s
full_file		2.29 s
symbol_oracle		2.13 s
line_span		2.12 s

Hard Task Accuracy

full_file		33.3%
bundle_oracle		33.3%
filtered_bundle		33.3%
symbol_oracle		0%
symbol_select		0%

Easy Task Prompt Tokens

full_repo		67,188 tokens
symbol_select		33,871 tokens
full_file		256 tokens
symbol_oracle		199 tokens
line_span		199 tokens

Selector Cost Improvement

old selector		33,922 tokens
filtered bundle		587 tokens

EXPERIMENT RESULTS 2

self_extend results

Self-Extend increased accepted context length, but did not recover passkey retrieval or code-assistance accuracy in this environment.

Compared modes

off
Default 4k attention without group attention.

G=2
Group attention with about 2x reported context.

G=4
Group attention with about 4x reported context.

passkey
Recover a hidden 5-digit key from long junk text.

code assistance
Recover function value, symbol reference, and test failure cause from long repo context.

PG-19 PPL
Baseline PPL only; group-attention flags were not available.

Passkey Success Rate

off		66.7%
G=2		0%
G=4		0%

Reported Context Capacity

G=4		16,608 ctx
G=2		8,416 ctx
off		4,320 ctx

Code Assistance Success

off		0%
G=2		0%
G=4		0%

PG-19 PPL Baseline

Q5 ctx4096		309 PPL
Q4 ctx4096		305 PPL
Q5 ctx2048		7.86 PPL
Q4 ctx2048		7.85 PPL

EXPERIMENT RESULTS 3

runtime_pointer results

Runtime pointer modes were accurate while keeping prompt size nearly constant as generated runtime objects grew.

Compared modes

full_runtime
Insert the entire producer output into the prompt.

runtime_pointer_oracle
Use the known correct extraction call.

runtime_pointer_select
Let the LLM select a tool call from runtime catalog and tool specs.

Medium: 3/3 success for pointer modes, 1/3 for full runtime. Prompt tokens fell from 131,136 to 757 for selected mode.

Medium Accuracy

pointer_oracle		100%
pointer_select		100%
full_runtime		33.3%

Medium Prompt Tokens

full_runtime		131,136 tokens
pointer_select		757 tokens
pointer_oracle		311 tokens

Medium Wall Time

full_runtime		26.27 s
pointer_select		6.89 s
pointer_oracle		2.61 s

EXPERIMENT RESULTS 4

source_pointer results

Source pointer modes avoided context overflow and removed large disk files from the LLM prompt.

Compared modes

full_source
Insert the whole log, markdown document, or JSON file into the prompt.

pointer_oracle
Use the known correct extraction call.

pointer_select
Let the LLM choose an extraction tool from a compact source catalog.

Medium: 3/3 success for pointer modes, 1/3 for full source. Prompt tokens fell from 120,208 to 780 for selected mode.

Medium Accuracy

pointer_oracle		100%
pointer_select		100%
full_source		33.3%

Medium Prompt Tokens

full_source		120,208 tokens
pointer_select		780 tokens
pointer_oracle		286 tokens

Medium Wall Time

full_source		22.12 s
pointer_select		6.42 s
pointer_oracle		2.44 s

SYNTHESIS

Final comparison and application plan

Approach	Best target	Experiment result	Use
symbol_index	repository code	Fast when bundled; old full-index selector was too expensive.	default
source_pointer	disk logs, docs, JSON	3/3 success with stable prompt size.	default
runtime_pointer	tool output and intermediate values	3/3 success with runtime size decoupled from prompt size.	default
self_extend	wide background context	Accepted longer input, but retrieval failed.	fallback

Representative Medium Prompt Sizes



Representative medium prompt sizes. Log scale.

CONCLUSION

Use retrieval first, length extension last

- 1

Default external evidence access to `source://` plus typed extraction tools.
- 2

Default tool-output reuse to `runtime://` plus RuntimeStore-backed mirrored tools.
- 3

Default code access to filtered symbol bundles, not full repository prompts.
- 4

Keep Self-Extend as an experimental fallback gated by retrieval sanity checks.

Immediate integration

Combine source pointer evidence, runtime pointer evidence, and symbol bundle context in one agent workflow.

Quality improvement

Add cross-file dependency expansion, deterministic execution for arithmetic/string tasks, and cached symbol bundles.

Future evaluation

Repeat Self-Extend with a build that exposes group attention consistently across perplexity, passkey, and code tasks.